Django Básico

Asimov Academy

ASIMOV

Conteúdo

Capítulo 1: O que é Django e por que utilizá-lo? 6
Introdução . . . 6
1.1 O que é Django? . . . 6
Definição . . . 6
1.2 Principais Características e Vantagens do Django . . . 6
Desenvolvimento Rápido . . . 6
Batteries Included ("Baterias Incluídas") . . . 6
Segurança Integrada . . . 7
Escalabilidade . . . 7
Ampla Comunidade e Documentação . . . 7
1.3 Casos de Uso do Django em Aplicações Reais . . . 7
O que você estudou: . . . 7

Capítulo 2: O que é um Framework? 9
Introdução . . . 9
2.1 O que é um Framework? . . . 9
2.2 Vantagens de Usar Frameworks . . . 9
1. Desenvolvimento mais rápido . . . 9
2. Segurança embutida . . . 10
3. Código organizado e reutilizável . . . 10
4. Comunidade ativa e suporte . . . 10
2.3 Tipos de Frameworks: Backend vs Frontend . . . 10
Frameworks Backend . . . 10
Frameworks Frontend . . . 10
2.4 O Django como Framework Backend . . . 11
O que você estudou: . . . 11

Capítulo 3: A Arquitetura MVT (Model-View-Template) 12
Introdução . . . 12
3.1 O que é a Arquitetura MVT? . . . 12
Definição e Objetivo . . . 12
3.2 Componentes da Arquitetura MVT . . . 12
Model (Modelo) . . . 12
View (Visão) . . . 12
Template (Modelo de Exibição) . . . 13
3.3 Como os Componentes MVT se Relacionam? . . . 13

3.4 Diferença entre MVT e MVC 14
Semelhanças entre MVT e MVC 14
Principais Diferenças 14
O que você estudou: 15

Capítulo 4: Instalação do Django **16**
Introdução 16
4.1 Preparação do Ambiente 16
Criando um Ambiente Virtual 16
4.2 Instalando o Django 17
4.3 Criando o Primeiro Projeto Django 17
4.4 Executando o Servidor de Desenvolvimento 17
O que você estudou: 18

Capítulo 5: Explorando a Estrutura de um Projeto Django **19**
Introdução 19
5.1 Estrutura de Pastas e Arquivos do Django 19
5.2 Explicação dos Arquivos Mais Importantes 19
`manage.py` 19
A Pasta `meuprojeto/` 20
O que você estudou: 21

Capítulo 6: Projetos e Apps no Django **22**
Introdução 22
6.1 O que é um Projeto Django? 22
Principais características de um projeto Django 22
6.2 O que é um App no Django? 22
Características de um app Django 22
Exemplo prático 23
6.3 Como um Projeto e um App se Relacionam? 23
Registrando um App no Projeto 23
O que você estudou: 23

Capítulo 7: Criando o Primeiro App **25**
Introdução 25
7.1 Criando o Primeiro App 25
Passo 1: Executar o comando 25
Passo 2: Entendendo os arquivos do app 25

7.2 Registrando o App no Projeto . . . 26
Passo a passo: . . . 26
O que você estudou: . . . 27

Capítulo 8: O que são Views no Django? **28**
Introdução . . . 28
8.1 O que é uma View? . . . 28
Resumo . . . 28
8.2 Criando uma View Simples . . . 28
Passo 1: Criar a View . . . 28
Explicação . . . 28
Passo 2: Mapear a View para uma URL . . . 29
Passo 3: Testar no Navegador . . . 29
8.3 Tornando a View Mais Dinâmica . . . 29
Nova View: Saudação Personalizada . . . 29
O que essa view faz? . . . 30
8.4 Explorando o Código . . . 30
HttpResponse . . . 30
request . . . 30
Mapeamento de URL . . . 30
O que você estudou: . . . 31
Desafio opcional . . . 31

Capítulo 9: URLs e Rotas no Django **32**
Introdução . . . 32
9.1 O que são URLs e Rotas? . . . 32
Conceito . . . 32
Analogia . . . 32
9.2 Como o Django Lida com URLs . . . 32
Exemplo de configuração básica em `urls.py` do projeto: . . . 32
9.3 Organizando URLs por App . . . 33
Passo 1: Criar um `urls.py` dentro do seu app (`meuapp/urls.py`) . . . 33
Passo 2: Conectar o `urls.py` do app ao projeto . . . 33
9.4 Explicando a Função `path()` . . . 33
Testando no Navegador . . . 34
O que você estudou: . . . 34

Capítulo 10: Views e URLs Dinâmicas **35**
Introdução . . . 35

10.1 Views com Parâmetros . . . 35
O que são parâmetros em views? . . . 35
Exemplo 1: Saudação personalizada . . . 35
10.2 URLs Dinâmicas com Múltiplos Parâmetros . . . 36
10.3 Tipos de Parâmetros em URLs . . . 36
10.4 Exemplo Prático: Detalhes de Produto . . . 36
O que você estudou: . . . 37

Capítulo 11: Renderizando Templates **38**
Introdução . . . 38
11.1 O que são Templates? . . . 38
Exemplo prático . . . 38
Benefícios dos templates no Django . . . 38
11.2 Estruturando a Pasta de Templates . . . 38
Passo 1: Criar a pasta `templates` . . . 38
11.3 Configurando o Django para Encontrar os Templates . . . 39
11.4 Criando e Renderizando um Template . . . 39
Passo 1: Criar o arquivo `index.html` . . . 39
Passo 2: Alterar a view para renderizar o template . . . 40
Passo 3: Testar no navegador . . . 40
11.5 Explicando os Elementos do Código . . . 40
O que você estudou: . . . 40

Capítulo 12: Sintaxe de Templates **41**
Introdução . . . 41
12.1 Utilizando Variáveis nos Templates . . . 41
Exemplo: Saudação personalizada . . . 41
No arquivo `home.html`: . . . 41
12.2 Estruturas de Controle no Template . . . 42
Condicional `if` . . . 42
Repetição com `for` . . . 42
12.3 Filtros no Template . . . 43
Exemplos de filtros úteis . . . 43
Boas práticas com templates . . . 43
O que você estudou: . . . 43

Capítulo 13: Herança de Templates **44**
Introdução . . . 44
13.1 Por que reutilizar templates? . . . 44

13.2 O que é herança de templates? 44
13.3 Configurando a pasta de templates no projeto 44
13.4 Criando o Template Base (`base.html`) 45
Passo 1: Estrutura da pasta 45
Passo 2: Conteúdo do `base.html` 45
Explicação 46
13.5 Criando um Template Filho 46
Exemplo: `index.html` 46
View correspondente (`views.py`) 46
Rota no `urls.py` 46
13.6 Exemplo com Iteração: `produtos.html` 46
View correspondente 47
13.7 Benefícios da Herança de Templates 47
O que você estudou: 47

Capítulo 1: O que é Django e por que utilizá-lo?

Introdução

O Django é um dos frameworks web mais populares para o desenvolvimento backend em Python. Ele foi projetado para facilitar a criação de aplicações web de forma rápida, segura e escalável. Neste capítulo, exploraremos suas principais características, vantagens e casos de uso em aplicações reais.

1.1 O que é Django?

Definição

O Django é um framework de desenvolvimento web de alto nível e de código aberto, escrito em Python. Seu principal objetivo é permitir o desenvolvimento rápido e eficiente de aplicações, automatizando tarefas comuns e permitindo que os desenvolvedores foquem na lógica do negócio.

Inicialmente criado para atender às demandas de sites de notícias — que precisam lidar com grandes volumes de dados, atualizações frequentes e alto nível de segurança —, o Django se tornou amplamente adotado em diversas áreas, incluindo redes sociais, e-commerce e plataformas educacionais.

1.2 Principais Características e Vantagens do Django

Desenvolvimento Rápido

O Django permite criar aplicações rapidamente, pois já inclui recursos prontos, como autenticação, administração e sistema de roteamento.

Batteries Included ("Baterias Incluídas")

Esse princípio significa que o Django já traz diversas funcionalidades integradas, eliminando a necessidade de ferramentas externas. Ele inclui:
- ORM (Object-Relational Mapping) para interagir com bancos de dados;
- Sistema de autenticação e controle de usuários;
- Gerenciamento de rotas e URLs;
- Suporte para envio de e-mails e manipulação de arquivos.

Segurança Integrada

A segurança é um dos pilares do Django. Ele protege automaticamente contra vulnerabilidades comuns, como:
SQL Injection – ataques que exploram falhas em consultas SQL;
Cross-Site Scripting (XSS) – injeção de scripts maliciosos em páginas web;
Cross-Site Request Forgery (CSRF) – ataques que exploram a autenticação do usuário para executar ações não autorizadas.

Escalabilidade

Empresas que necessitam de aplicações robustas e que suportem um grande número de usuários confiam no Django. Ele é amplamente utilizado em redes sociais e plataformas de e-commerce, onde a demanda por desempenho e estabilidade é alta.

Ampla Comunidade e Documentação

A grande popularidade do Django garantiu uma comunidade ativa e uma excelente documentação. Isso facilita o aprendizado e a resolução de dúvidas, tornando o desenvolvimento mais acessível e colaborativo.

1.3 Casos de Uso do Django em Aplicações Reais

Redes Sociais: Plataformas como Instagram e Pinterest utilizam Django para lidar com milhões de usuários e interações simultâneas.

E-commerce: Lojas online utilizam Django devido à sua segurança e flexibilidade. Um exemplo é o OpenStack.

Sites Institucionais e Blogs: O Django permite criar rapidamente sites informativos e blogs, aproveitando seu sistema de administração embutido.

APIs e Integrações: Empresas utilizam Django para desenvolver APIs RESTful, permitindo a comunicação entre diferentes sistemas e aplicativos.

O que você estudou:

- O que é o Django e por que ele é amplamente utilizado;
- As principais características do Django, incluindo rapidez, segurança e escalabilidade;

- O conceito de “Batteries Included” e como o Django oferece um conjunto de ferramentas integradas;
- Exemplos de casos de uso do Django em aplicações reais.

Capítulo 2: O que é um Framework?

Introdução

Desenvolver uma aplicação web envolve diversas etapas, como estruturar um banco de dados, gerenciar autenticação, lidar com requisições e respostas, além de organizar os arquivos do projeto.

Os frameworks surgiram para facilitar esse processo, oferecendo uma estrutura pré-definida e um conjunto de ferramentas que padronizam a construção das aplicações. Isso permite que os desenvolvedores foquem na lógica do projeto, sem precisar reescrever código repetitivo ou lidar com configurações complexas.

Neste capítulo, vamos entender o que são frameworks, suas vantagens e as diferenças entre frameworks de backend e frontend.

2.1 O que é um Framework?

Um **framework** é um conjunto de ferramentas, bibliotecas e padrões que facilitam o desenvolvimento de aplicações. Ele fornece uma estrutura pronta, permitindo que os desenvolvedores criem software de forma mais eficiente.

Imagine que você quer montar uma mesa:

- Você pode comprar tábuas de madeira, serrar, lixar e montar tudo sozinho. Isso exige muito esforço e tempo.

- Ou você pode comprar um kit pronto, onde todas as peças já vêm cortadas, com os parafusos certos e um manual explicando o passo a passo.

Esse kit é como um framework: ele fornece os blocos essenciais para construir sua aplicação sem precisar reinventar a roda. O desenvolvedor ainda tem liberdade para personalizar, mas já começa com uma base estruturada.

2.2 Vantagens de Usar Frameworks

1. Desenvolvimento mais rápido

Com um framework, várias funções e componentes já estão prontos, permitindo que o desenvolvedor se concentre na lógica da aplicação em vez de reescrever funcionalidades básicas.

2. Segurança embutida

Frameworks populares incluem proteções contra vulnerabilidades comuns, como injeção de SQL e ataques XSS, reduzindo riscos de segurança.

3. Código organizado e reutilizável

A estrutura imposta pelos frameworks ajuda a manter um código limpo e modular, facilitando a manutenção e o trabalho em equipe.

4. Comunidade ativa e suporte

Frameworks amplamente utilizados contam com grandes comunidades, documentação extensa e muitos tutoriais disponíveis, facilitando a resolução de problemas.

2.3 Tipos de Frameworks: Backend vs Frontend

Os frameworks podem ser classificados em **backend** e **frontend**, de acordo com a parte do sistema que auxiliam no desenvolvimento.

Frameworks Backend

Os frameworks backend são responsáveis pela lógica da aplicação, ou seja, tudo que acontece no servidor. Eles lidam com a comunicação com bancos de dados, autenticação de usuários, segurança e regras de negócio.

Exemplos de frameworks backend:
- **Django (Python)** – Focado em produtividade e segurança.
- **Flask (Python)** – Mais leve e flexível.
- **Express.js (JavaScript)** – Utilizado com Node.js para aplicações web.
- **Spring Boot (Java)** – Muito usado em aplicações empresariais.

Frameworks Frontend

Os frameworks frontend lidam com a interface visual e a experiência do usuário. Eles estruturam o que é exibido na tela e facilitam a criação de interfaces dinâmicas.

Exemplos de frameworks frontend:
- **React.js (JavaScript)** – Criado pelo Facebook, usado para interfaces interativas.
- **Vue.js (JavaScript)** – Simples e flexível para criação de interfaces.
- **Angular (JavaScript)** – Framework completo para grandes aplicações web.

2.4 O Django como Framework Backend

Agora que entendemos o conceito de frameworks, vamos focar no **Django**, um framework backend escrito em Python.

O Django foi projetado para desenvolvimento rápido e seguro de aplicações web, oferecendo um conjunto de ferramentas completas para criar sistemas robustos. Ele inclui:

- Gerenciamento de banco de dados com ORM (Object-Relational Mapping);
- Sistema de autenticação de usuários integrado;
- Proteções de segurança contra ataques comuns;
- Templates para renderizar páginas HTML;
- Um servidor de desenvolvimento pronto para testes.

Essa estrutura permite que desenvolvedores criem aplicações de maneira mais ágil e organizada.

O que você estudou:

- O que é um framework e sua função no desenvolvimento de software;
- As vantagens de utilizar um framework, como agilidade, segurança e organização do código;
- A diferença entre frameworks **backend** e **frontend**;
- A importância do Django como um framework backend para desenvolvimento web.

Capítulo 3: A Arquitetura MVT (Model-View-Template)

Introdução

Neste capítulo, vamos entender como o Django organiza o desenvolvimento de aplicações utilizando a arquitetura MVT. Esse modelo facilita a criação e manutenção de projetos, separando as responsabilidades entre Model (Modelo), View (Visão) e Template.

3.1 O que é a Arquitetura MVT?

Definição e Objetivo

A arquitetura **MVT (Model-View-Template)** é o padrão adotado pelo Django para estruturar aplicações web. Seu objetivo principal é organizar o código em camadas bem definidas, garantindo modularidade e facilitando a manutenção e escalabilidade da aplicação.

Esse modelo é uma variação do **MVC (Model-View-Controller)**, um dos padrões mais utilizados no desenvolvimento de software. No MVC, o **Controller** gerencia a comunicação entre as camadas. Já no MVT, o Django assume esse papel internamente, simplificando o fluxo da aplicação.

A separação de responsabilidades no MVT permite que diferentes partes do sistema sejam desenvolvidas de forma independente. Por exemplo, enquanto um desenvolvedor trabalha na modelagem dos dados, outro pode criar a interface da aplicação sem interferências.

3.2 Componentes da Arquitetura MVT

Model (Modelo)

O **Model** é responsável pela estrutura e manipulação dos dados da aplicação. No Django, ele representa as tabelas do banco de dados e define quais informações serão armazenadas.

O Django utiliza um sistema chamado **ORM (Object-Relational Mapping)**, que permite interagir com o banco de dados utilizando código Python, sem a necessidade de escrever consultas SQL manualmente. Isso torna o desenvolvimento mais eficiente e seguro.

View (Visão)

A **View** é responsável pela lógica de negócios da aplicação. Ela processa as requisições dos usuários, interage com o Model e retorna uma resposta. Essa resposta pode ser:

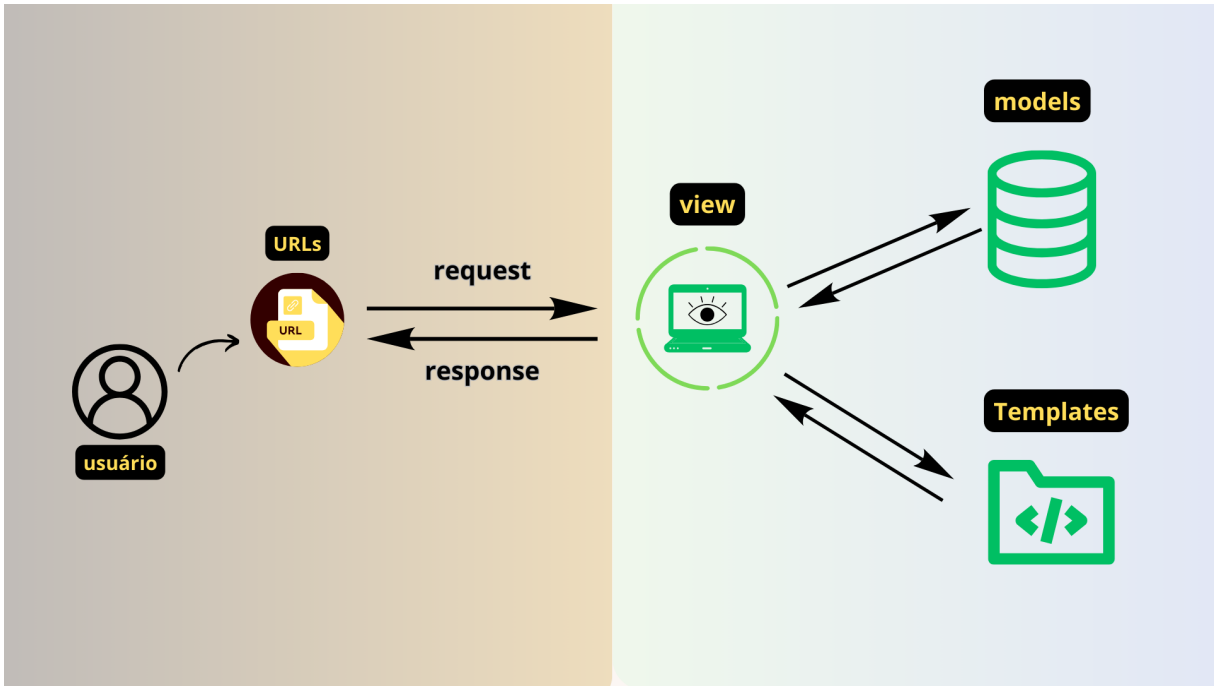
- Um template HTML renderizado para exibição no navegador;
- Um dado em formato JSON para consumo por APIs;
- Um simples texto como resposta à requisição.

Template (Modelo de Exibição)

O **Template** define a interface visual da aplicação. Ele contém arquivos HTML que estruturam o layout da página e permitem a exibição dinâmica dos dados fornecidos pela View.

O Django usa a **linguagem de templates**, que permite incluir variáveis dinâmicas dentro de `{{ }}` e utilizar estruturas condicionais e loops para organizar o conteúdo.

3.3 Como os Componentes MVT se Relacionam?



O fluxo básico de dados no modelo MVT funciona da seguinte forma:

1. O usuário faz uma requisição no navegador, acessando uma URL.
2. O Django encaminha a requisição para uma **View** específica.
3. A **View** interage com o **Model** para buscar ou processar dados.
4. A **View** envia os dados para o **Template**.
5. O **Template** renderiza o HTML final e o retorna ao navegador do usuário.

Esse fluxo organiza o desenvolvimento de maneira estruturada, separando cada responsabilidade no código.

3.4 Diferença entre MVT e MVC

Ambos os padrões seguem a mesma ideia: **separar a aplicação em camadas distintas** para organizar o código e facilitar a manutenção. No entanto, há diferenças importantes entre eles.

Semelhanças entre MVT e MVC

- Ambos dividem a aplicação em três camadas principais: Model, View e a interface com o usuário.
- No Django, o **Model** é equivalente ao do MVC tradicional e usa o ORM para interagir com o banco de dados.

Principais Diferenças

Característica	MVC	MVT (Django)
Controller	Existe e precisa ser definido manualmente pelo desenvolvedor.	O Django gerencia essa parte internamente, eliminando a necessidade de um Controller explícito.
View	Responsável pela lógica e apresentação dos dados.	Apenas processa a lógica e envia os dados para o Template.
Template	Pode estar integrado à View.	Totalmente separado, responsável apenas pela exibição.

No **MVC**, o desenvolvedor precisa definir um **Controller** que processa as requisições e interage com o Model e a View. Já no **MVT**, essa parte é gerenciada automaticamente pelo Django, simplificando o desenvolvimento.

Outra diferença importante é a ênfase na camada de **Template** no MVT. Enquanto no MVC a View pode lidar com a lógica de exibição, no MVT essa responsabilidade é isolada nos templates, tornando o código mais modular.

O que você estudou:

- Como o Django organiza o código utilizando a arquitetura MVT;
- O papel de cada componente: **Model**, **View** e **Template**;
- Como esses componentes se relacionam no fluxo de dados de uma aplicação Django;
- As principais diferenças entre **MVT** e **MVC**.

Capítulo 4: Instalação do Django

Introdução

Neste capítulo, daremos os primeiros passos práticos com o Django. Vamos aprender como instalar o framework em nosso ambiente de desenvolvimento, garantindo que tudo esteja configurado corretamente.

4.1 Preparação do Ambiente

Antes de instalar o Django, é importante criar um ambiente virtual. Essa prática permite isolar os pacotes do projeto, evitando conflitos entre diferentes versões de bibliotecas.

> **O que é um ambiente virtual?**
> Um ambiente virtual é uma área isolada do sistema onde os pacotes Python são instalados separadamente. Isso garante que cada projeto tenha suas próprias dependências, sem interferir em outros projetos.

Criando um Ambiente Virtual

1. Crie uma pasta para o projeto:

   ```
   mkdir primeiro_projeto
   cd primeiro_projeto
   ```

2. Dentro da pasta, crie o ambiente virtual:

   ```
   python -m venv meu_ambiente
   ```

   Isso criará uma pasta chamada `meu_ambiente` com os arquivos necessários para o isolamento do ambiente.

3. Ative o ambiente virtual:

   - No Linux/macOS:

     ```
     source meu_ambiente/bin/activate
     ```

   - No Windows:

     ```
     meu_ambiente\Scripts\activate
     ```

Agora o ambiente virtual está pronto para receber a instalação do Django.

4.2 Instalando o Django

Com o ambiente virtual ativado, utilizamos o `pip` (gerenciador de pacotes do Python) para instalar o Django:

```
pip install django
```

Para verificar se a instalação foi concluída com sucesso, execute:

```
django-admin --version
```

Caso seja necessário instalar uma versão específica do Django, utilize:

```
pip install django==4.2
```

A versão pode variar conforme os requisitos do projeto.

4.3 Criando o Primeiro Projeto Django

Agora que o Django está instalado, podemos criar nosso primeiro projeto. O comando para isso é:

```
django-admin startproject meuprojeto .
```

Esse comando cria a estrutura inicial do projeto, contendo os arquivos e diretórios essenciais.

4.4 Executando o Servidor de Desenvolvimento

O Django inclui um servidor de desenvolvimento que permite testar a aplicação localmente. Para iniciá-lo, utilize:

```
python manage.py runserver
```

Ao executar esse comando, o terminal exibirá uma saída semelhante a esta:

```
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
February 7, 2025 - 10:15:23
Django version 4.2, using settings 'meuprojeto.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CONTROL-C.
```

Acesse o endereço `http://127.0.0.1:8000/` no navegador. Se tudo estiver configurado corretamente, será exibida a página padrão do Django.

Para parar o servidor, pressione **Ctrl + C** no terminal.

O que você estudou:

- Como configurar um ambiente virtual para projetos Django;
- Como instalar o Django utilizando o `pip`;
- Como criar um projeto Django e iniciar o servidor de desenvolvimento;
- Como acessar a página inicial do Django e testar se a instalação foi bem-sucedida.

Capítulo 5: Explorando a Estrutura de um Projeto Django

Introdução

Agora que já criamos nosso primeiro projeto Django e executamos o servidor, é importante entender a estrutura do projeto gerada automaticamente. O Django organiza os arquivos de forma padronizada, facilitando o desenvolvimento e a manutenção da aplicação. Neste capítulo, vamos explorar as pastas e arquivos essenciais do projeto.

5.1 Estrutura de Pastas e Arquivos do Django

Ao rodar o comando:

```
django-admin startproject meuprojeto
```

O Django cria a seguinte estrutura de diretórios e arquivos:

```
primeiro_projeto/
├── manage.py
├── meuprojeto/
│   ├── __init__.py
│   ├── settings.py
│   ├── urls.py
│   ├── asgi.py
│   ├── wsgi.py
```

Cada um desses arquivos tem um papel fundamental no funcionamento do projeto. Vamos analisá-los em detalhes.

5.2 Explicação dos Arquivos Mais Importantes

`manage.py`

Este é um dos arquivos principais do Django. Ele funciona como uma ferramenta de linha de comando que permite gerenciar o projeto.

Com o `manage.py`, podemos executar diversos comandos, como:
- **Iniciar o servidor de desenvolvimento:**

```
python manage.py runserver
```

- **Criar aplicativos dentro do projeto:**

  ```
  python manage.py startapp nome_do_app
  ```

- **Gerenciar o banco de dados:**

  ```
  python manage.py migrate
  ```

A Pasta `meuprojeto/`

Essa pasta contém os arquivos principais de configuração do projeto.

`settings.py` Este arquivo contém todas as configurações do projeto, incluindo:
- Configuração do banco de dados;
- Idioma e fuso horário;
- Aplicativos instalados no Django (`INSTALLED_APPS`);
- Configurações de segurança e autenticação.

Sempre que for necessário ativar um novo aplicativo ou modificar o comportamento do projeto, esse arquivo será ajustado.

`urls.py` Responsável por definir as rotas da aplicação. Sempre que uma nova página for criada, é necessário registrar uma URL neste arquivo.

Exemplo de uma URL simples no Django:

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.home, name='home'),
]
```

`__init__.py` Indica que a pasta `meuprojeto/` é um **pacote Python**. Esse arquivo permite que o Django reconheça o diretório como parte do projeto.

`asgi.py` e `wsgi.py` Esses arquivos são utilizados para a execução do projeto em servidores web:
- **`wsgi.py`**: Usado em servidores tradicionais.
- **`asgi.py`**: Permite suporte a aplicações assíncronas.

Durante o desenvolvimento local, esses arquivos não precisam ser modificados.

O que você estudou:

- Como o Django organiza a estrutura de um projeto;
- O papel do arquivo `manage.py` na administração do projeto;
- A função dos arquivos `settings.py`, `urls.py`, `__init__.py`, `asgi.py` e `wsgi.py`;
- Como definir rotas no arquivo `urls.py`.

Capítulo 6: Projetos e Apps no Django

Introdução

Agora que já entendemos o que é o Django e como ele organiza um projeto, chegou o momento de esclarecer uma dúvida comum entre iniciantes: **qual a diferença entre um projeto e um app no Django?**

Embora esses dois conceitos possam parecer semelhantes, compreender essa separação é essencial para estruturar corretamente uma aplicação Django. Neste capítulo, vamos explorar o que é um **projeto**, o que é um **app**, como eles se relacionam e por que essa divisão facilita o desenvolvimento.

6.1 O que é um Projeto Django?

O **projeto Django** é a estrutura principal que reúne todas as configurações e arquivos essenciais para uma aplicação. Ele funciona como um **container** para um ou mais aplicativos dentro do Django.

Principais características de um projeto Django

- Contém **configurações globais**, como conexão com o banco de dados, idioma e fuso horário.
- Gerencia **as rotas principais da aplicação** por meio do arquivo `urls.py`.
- Define **quais apps fazem parte do projeto**, registrando-os no arquivo `settings.py`.

6.2 O que é um App no Django?

Os **apps** são módulos dentro de um projeto Django, responsáveis por funcionalidades específicas. Um projeto pode conter vários apps, cada um gerenciando uma parte independente do sistema.

Características de um app Django

- Representa uma funcionalidade dentro do sistema.
- Pode ser reutilizado em diferentes projetos.
- Trabalha de forma independente, mas pode se comunicar com outros apps dentro do mesmo projeto.

Exemplo prático

Imagine que estamos criando um **site de e-commerce**. Podemos organizar nossa aplicação Django da seguinte forma:
- Um **app "produtos"** para cadastrar e exibir os produtos.
- Um **app "pedidos"** para gerenciar compras e carrinhos de compra.
- Um **app "usuarios"** para lidar com cadastro, login e perfis de clientes.

Cada um desses apps pode funcionar separadamente dentro do projeto, tornando a aplicação mais modular e organizada.

6.3 Como um Projeto e um App se Relacionam?

Agora que sabemos que um projeto Django pode ter vários apps, como eles se conectam?

Registrando um App no Projeto

Os apps são criados dentro do projeto, mas precisam ser registrados para que o Django os reconheça. Isso é feito no arquivo `settings.py`, dentro da variável `INSTALLED_APPS`.

Exemplo de um app chamado `usuarios` registrado no projeto:

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'usuarios',  # App criado para gerenciar usuários
]
```

Após esse registro, o app passa a ser reconhecido pelo Django e pode ser utilizado normalmente.

Essa separação entre **projeto** e **app** permite que um mesmo app possa ser reaproveitado em diferentes projetos. Se tivermos um app de **autenticação de usuários**, por exemplo, poderíamos usá-lo tanto em um site de e-commerce quanto em uma rede social sem precisar recriar tudo do zero.

O que você estudou:

- O **projeto Django** é a estrutura principal da aplicação, responsável pelas configurações globais;

- Um **app Django** é um módulo dentro do projeto, responsável por uma funcionalidade específica;
- Um **projeto pode conter vários apps**, tornando a aplicação mais modular e organizada;
- A separação entre projeto e app facilita a manutenção e a reutilização de código.

Capítulo 7: Criando o Primeiro App

Introdução

Nos capítulos anteriores, aprendemos o que é um projeto Django, exploramos sua estrutura de arquivos e entendemos a separação entre projeto e app. Agora, chegou o momento de colocarmos isso em prática: vamos **criar nosso primeiro app** dentro de um projeto Django.

Um app no Django é um módulo que executa uma função específica dentro de um sistema maior. É assim que o Django organiza o desenvolvimento de aplicações complexas, dividindo a lógica em partes reutilizáveis e independentes.

7.1 Criando o Primeiro App

Para criar um app dentro do seu projeto, siga os passos abaixo:

Passo 1: Executar o comando

No terminal, dentro da pasta do seu projeto, digite:

```
python manage.py startapp meuapp
```

Esse comando irá criar uma nova pasta chamada `meuapp/`, com a seguinte estrutura de arquivos:

```
meuapp/
├── migrations/
├── __init__.py
├── admin.py
├── apps.py
├── models.py
├── tests.py
├── views.py
```

Passo 2: Entendendo os arquivos do app

Cada um desses arquivos tem uma função importante no desenvolvimento da aplicação:

- **`models.py`**: Onde definimos os modelos do banco de dados. Cada classe criada aqui representa uma tabela na base de dados.

- **`views.py`**: Contém a lógica de controle da aplicação. Ele define como o sistema responde às requisições feitas pelo usuário.

- **`admin.py`**: Usado para configurar o painel administrativo do Django.

- **`migrations/`**: Armazena os arquivos de migração, que são responsáveis por criar e alterar estruturas no banco de dados com base nos modelos definidos.

Outros arquivos como `apps.py`, `tests.py` e `__init__.py` também fazem parte da estrutura básica, mas os principais para início do desenvolvimento são os listados acima.

7.2 Registrando o App no Projeto

Criar o app não é suficiente. Para que o Django reconheça que ele faz parte do projeto, precisamos **registrá-lo no arquivo de configurações `settings.py`**, mais especificamente dentro da variável `INSTALLED_APPS`.

Passo a passo:

1. Abra o arquivo `settings.py`, localizado dentro da pasta do projeto (ex: `meuprojeto/settings.py`).
2. Encontre a seção `INSTALLED_APPS`, que normalmente se parece com isto:

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
]
```

3. Adicione o nome do seu app (`meuapp`) à lista:

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'meuapp',  # App criado pelo usuário
]
```

> Ao fazer isso, o Django passa a reconhecer o app como parte oficial do projeto, o que permite a criação de modelos, views, URLs, templates e outras funcionalidades.

O que você estudou:

- O que é um app dentro de um projeto Django
- Como criar um app utilizando o comando `startapp`
- A função de cada arquivo na estrutura de um app
- Como registrar um app no Django através do arquivo `settings.py`

Capítulo 8: O que são Views no Django?

Introdução

Dentro da arquitetura **MVT** do Django (Model-View-Template), a camada **View** é responsável por lidar com a lógica de processamento das requisições. Sempre que um usuário acessa uma página do seu site, é a view que decide qual resposta será enviada de volta.

Neste capítulo, você vai entender: - O que são views no Django; - Como criar e mapear views simples; - Como exibir respostas dinâmicas para o usuário.

8.1 O que é uma View?

No Django, uma **view** é uma função (ou classe) que: - Recebe uma requisição HTTP do navegador (por meio da URL acessada); - Processa essa requisição (por exemplo, acessando o banco de dados, calculando algo, etc.); - Retorna uma resposta (normalmente HTML ou texto simples).

Resumo

> Uma view conecta a ação do usuário à lógica da aplicação. É ela que "decide" o que será exibido quando o usuário acessa uma determinada página.

8.2 Criando uma View Simples

Vamos criar uma view básica que retorna a mensagem "Olá, Mundo!" no navegador.

Passo 1: Criar a View

Abra o arquivo `views.py` dentro do seu app (por exemplo, `meuapp/views.py`) e adicione o seguinte código:

```
from django.http import HttpResponse

def home(request):
    return HttpResponse("Olá, Mundo!")
```

Explicação

- **`HttpResponse`**: classe do Django usada para retornar uma resposta HTTP.

- **`request`**: parâmetro obrigatório, que representa a requisição feita pelo usuário.
- **`home`**: nome da função que definimos como view.

Passo 2: Mapear a View para uma URL

Agora, precisamos indicar ao Django qual URL vai chamar essa view.

1. Abra o arquivo `urls.py` do projeto (exemplo: `meuprojeto/urls.py`);
2. Importe a função home da sua view;
3. Crie o mapeamento:

```
from django.urls import path
from meuapp.views import home

urlpatterns = [
    path('', home, name='home'),  # URL raiz
]
```

Com isso, ao acessar `http://127.0.0.1:8000/`, a função home() será executada e mostrará a mensagem "Olá, Mundo!".

Passo 3: Testar no Navegador

1. Inicie o servidor de desenvolvimento:

   ```
   python manage.py runserver
   ```

2. Abra o navegador e acesse:

   `http://127.0.0.1:8000/`

Você verá a mensagem **Olá, Mundo!** na tela.

8.3 Tornando a View Mais Dinâmica

Vamos modificar a view home para exibir uma saudação diferente de acordo com o horário em que o usuário acessa o site.

Nova View: Saudação Personalizada

Substitua o conteúdo da função home por:

```
from django.http import HttpResponse
from datetime import datetime

def home(request):
    hora_atual = datetime.now().hour

    if 5 <= hora_atual < 12:
        mensagem = "Bom dia!"
    elif 12 <= hora_atual < 18:
        mensagem = "Boa tarde!"
    else:
        mensagem = "Boa noite!"

    return HttpResponse(mensagem)
```

O que essa view faz?

- Obtém o horário atual do sistema;
- Define a mensagem com base nesse horário;
- Retorna a mensagem para o navegador usando `HttpResponse`.

Exemplo de resultado: - Se o usuário acessar pela manhã: "Bom dia!" - À tarde: "Boa tarde!" - À noite: "Boa noite!"

> **Importante:** a URL continua sendo a raiz (`http://127.0.0.1:8000/`), pois usamos a mesma função home.

8.4 Explorando o Código

HttpResponse

É usada para retornar qualquer tipo de resposta textual. Pode ser HTML, JSON, ou texto puro.

request

O objeto que representa a requisição HTTP. Pode conter informações como: - Método da requisição (GET, POST); - Parâmetros; - Cabeçalhos HTTP.

Mapeamento de URL

No Django, associamos views a URLs com o auxílio da função `path()`. Dessa forma, cada página do seu site pode chamar uma view diferente, conforme a URL acessada.

O que você estudou:

- O conceito de view no Django e seu papel na arquitetura MVT;
- Como criar uma view simples que retorna texto;
- Como mapear essa view para uma URL;
- Como exibir mensagens diferentes com base em condições (ex: horário do dia).

Desafio opcional

Crie uma nova view chamada `bem_vindo` que exibe a mensagem:
"Bem-vindo ao meu site!"

Mapeie essa view para a URL `/bem-vindo/`.

Capítulo 9: URLs e Rotas no Django

Introdução

No capítulo anterior, aprendemos como criar **views**, que são responsáveis por processar requisições e retornar respostas ao usuário. Neste capítulo, vamos avançar um passo importante: entender como essas views são acessadas por meio das **URLs**.

As **URLs** são os "endereços" que digitamos no navegador, como `/`, `/contato/` ou `/sobre/`. Para que o Django saiba qual conteúdo exibir quando uma URL é acessada, usamos um sistema de **rotas**, que mapeia essas URLs para funções ou classes (views).

9.1 O que são URLs e Rotas?

Conceito

- A **URL (Uniform Resource Locator)** é o endereço que digitamos no navegador para acessar uma página da web.
- A **rota** é a instrução que diz ao Django: "Quando essa URL for acessada, execute esta view".

Analogia

Pense na URL como o endereço de uma casa, e na rota como o portão certo que leva até ela. O Django precisa saber por onde entrar para entregar o conteúdo solicitado.

9.2 Como o Django Lida com URLs

No Django, as URLs são configuradas em arquivos chamados `urls.py`. Esses arquivos contêm a relação entre o caminho da URL e a view que deve ser chamada quando esse caminho for acessado.

Exemplo de configuração básica em `urls.py` do projeto:

```
from django.contrib import admin
from django.urls import path
from meuapp import views

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', views.home, name='home'),  # Rota para a página inicial
]
```

Neste exemplo: - A URL `/admin/` leva ao painel administrativo do Django (configuração padrão). - A URL `/` (raiz do site) está mapeada para a função `home` da view localizada em `meuapp/views.py`.

9.3 Organizando URLs por App

À medida que o projeto cresce, manter todas as rotas no `urls.py` do projeto pode gerar confusão. Por isso, é boa prática criar um `urls.py` separado para cada **app** dentro do projeto.

Passo 1: Criar um `urls.py` dentro do seu app (`meuapp/urls.py`)

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.home, name='home'),
]
```

Neste exemplo: - A URL `''` representa a raiz do app `meuapp`. - A função `home` será chamada quando alguém acessar `/meuapp/`.

Passo 2: Conectar o `urls.py` do app ao projeto

Abra o arquivo `urls.py` do projeto (`meuprojeto/urls.py`) e adicione a instrução para incluir as rotas do app:

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('meuapp/', include('meuapp.urls')),
]
```

Com isso: - Qualquer URL que começar com `/meuapp/` será encaminhada para o arquivo `meuapp/urls.py`. - Essa separação torna o projeto mais modular e organizado.

9.4 Explicando a Função `path()`

A função `path()` é usada para definir cada rota. Ela aceita três argumentos principais:

```
path('caminho/', views.minha_view, name='nome_da_rota')
```

- **`'caminho/'`**: A parte da URL que será acessada.

- **`views.minha_view`**: A função (ou classe) que será chamada para processar a requisição.
- **`name='nome_da_rota'`** (opcional): Um nome interno que podemos usar em templates ou redirecionamentos.

Testando no Navegador

1. Inicie o servidor com:

   ```
   python manage.py runserver
   ```

2. Acesse a URL no navegador, por exemplo:

   ```
   http://127.0.0.1:8000/meuapp/
   ```

Você verá a resposta configurada pela sua view.

O que você estudou:

- O que são URLs e rotas e qual seu papel em uma aplicação Django;
- Como mapear URLs para views usando o arquivo `urls.py`;
- A estrutura recomendada para organizar rotas em projetos com múltiplos apps;
- Como usar `include()` para integrar o arquivo de rotas de um app ao projeto principal.

Capítulo 10: Views e URLs Dinâmicas

Introdução

Nos capítulos anteriores, você aprendeu a criar views simples e configurar URLs estáticas para acessá-las. Neste capítulo, vamos evoluir para algo mais poderoso e flexível: **views com parâmetros** e **URLs dinâmicas**.

Esse recurso permite criar páginas personalizadas com base em informações passadas diretamente na URL, como o nome de um usuário ou o código de um produto. Com isso, você conseguirá montar aplicações mais interativas, como perfis, páginas de detalhes e muito mais.

10.1 Views com Parâmetros

O que são parâmetros em views?

Parâmetros são informações que podem ser passadas por meio da URL para a view. Com isso, a resposta gerada pode ser adaptada de acordo com o conteúdo acessado.

Exemplo 1: Saudação personalizada

Vamos criar uma view que receba um nome na URL e exiba uma mensagem de boas-vindas:

```
from django.http import HttpResponse

def saudacao(request, nome):
    mensagem = f"Olá, {nome}! Bem-vindo ao nosso site."
    return HttpResponse(mensagem)
```

Para que essa view funcione corretamente, precisamos configurá-la no `urls.py`:

```
from django.urls import path
from . import views

urlpatterns = [
    path('saudacao/<str:nome>/', views.saudacao),
]
```

Agora, ao acessar `http://127.0.0.1:8000/saudacao/Ana/`, a resposta será:

```
Olá, Ana! Bem-vindo ao nosso site.
```

> Note que `<str:nome>` é um parâmetro dinâmico. Ele aceita qualquer texto e envia para a view como argumento.

10.2 URLs Dinâmicas com Múltiplos Parâmetros

Também é possível passar mais de um parâmetro na URL. Veja o exemplo a seguir:

```
from django.http import HttpResponse

def perfil_usuario(request, nome, idade):
    mensagem = f"Nome: {nome}, Idade: {idade} anos."
    return HttpResponse(mensagem)
```

E no `urls.py`:

```
path('perfil/<str:nome>/<int:idade>/', views.perfil_usuario),
```

Ao acessar `http://127.0.0.1:8000/perfil/Ana/25/`, a view retornará:

```
Nome: Ana, Idade: 25 anos.
```

10.3 Tipos de Parâmetros em URLs

O Django permite definir o tipo dos dados que serão capturados na URL:

Sintaxe	Tipo de dado capturado
`<str:nome>`	Texto (string)
`<int:id>`	Número inteiro
`<slug:slug>`	Slug (ex: textos amigáveis)
`<uuid:id>`	Identificador UUID
`<path:rota>`	Caminhos com / inclusos

10.4 Exemplo Prático: Detalhes de Produto

Vamos simular um cenário em que o usuário acessa detalhes de um produto específico, com base em seu ID:

```
def produto(request, id_produto):
    produtos = {
        1: "Notebook",
        2: "Teclado",
        3: "Mouse",
    }
    produto = produtos.get(id_produto, "Produto não encontrado")
    return HttpResponse(f"Detalhes do Produto: {produto}")
```

No arquivo `urls.py`, adicione a rota:

```
path('produto/<int:id_produto>/', views.produto),
```

Assim, ao acessar `http://127.0.0.1:8000/produto/2/`, será exibido:

```
Detalhes do Produto: Teclado
```

E ao acessar um ID inexistente, como `http://127.0.0.1:8000/produto/99/`, será exibido:

```
Detalhes do Produto: Produto não encontrado
```

O que você estudou:

- Como passar informações via URL para personalizar a resposta da view;
- Como configurar parâmetros dinâmicos no arquivo `urls.py`;
- Como utilizar múltiplos parâmetros em uma mesma rota;
- Como aplicar esse conhecimento em exemplos práticos, como exibir saudações e detalhes de produtos.

Capítulo 11: Renderizando Templates

Introdução

Ao desenvolvermos aplicações web, é fundamental separar a lógica da apresentação visual. No Django, essa separação é feita através dos **templates**, que representam a camada "T" da arquitetura MVT (Model-View-Template).

Templates são arquivos HTML dinâmicos que recebem dados das views e os exibem ao usuário. Neste capítulo, vamos entender: - O que são templates; - Como configurá-los no Django; - Como renderizar um template a partir de uma view.

11.1 O que são Templates?

Templates são arquivos HTML que contêm **variáveis** e **estruturas de controle**, como loops e condicionais. Eles são responsáveis pela apresentação visual dos dados processados no backend.

Exemplo prático

Imagine uma view que busca produtos no banco de dados. O template é responsável por exibir esses produtos em uma página HTML, com layout e design definidos.

Benefícios dos templates no Django

- **Separação entre lógica e apresentação**: as views lidam com o processamento, e os templates com a exibição.
- **Reutilização de código**: com a herança de templates, é possível definir um layout base e reaproveitá-lo em diversas páginas.
- **Facilidade de manutenção**: ao alterar um layout, não é necessário modificar várias páginas individualmente — basta editar o template base.

11.2 Estruturando a Pasta de Templates

Passo 1: Criar a pasta `templates`

Por padrão, o Django procura pelos templates em uma pasta chamada `templates`. O ideal é criar essa pasta **dentro do seu app**.

Exemplo de estrutura:

```
meuapp/
├── templates/
│   └── meuapp/
│       └── index.html
```

> Dica: usar o nome do app como subpasta ajuda a evitar conflitos entre templates de diferentes apps.

11.3 Configurando o Django para Encontrar os Templates

No arquivo `settings.py`, localize a variável `TEMPLATES` e adicione o caminho da pasta de templates no parâmetro `DIRS`:

```
import os

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [os.path.join(BASE_DIR, 'meuapp/templates')],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                # ...
            ],
        },
    },
]
```

Essa configuração garante que o Django saiba onde buscar os arquivos HTML a serem renderizados.

11.4 Criando e Renderizando um Template

Passo 1: Criar o arquivo `index.html`

Dentro da pasta `templates/meuapp/`, crie o arquivo `index.html` com o seguinte conteúdo:

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
    <meta charset="UTF-8">
    <title>Página Inicial</title>
</head>
<body>
    <h1>Bem-vindo ao meu site!</h1>
    <p>Este é o meu primeiro template no Django.</p>
</body>
</html>
```

Passo 2: Alterar a view para renderizar o template

No arquivo `views.py`, importe a função `render` e utilize-a:

```
from django.shortcuts import render

def home(request):
    return render(request, 'meuapp/index.html')
```

A função `render()` combina o template com os dados fornecidos pela view e retorna uma resposta HTTP.

Passo 3: Testar no navegador

1. Inicie o servidor:

   ```
   python manage.py runserver
   ```

2. Acesse a URL configurada para a view home. Você verá a página HTML definida no template `index.html`.

11.5 Explicando os Elementos do Código

- **`render(request, 'meuapp/index.html')`**: informa ao Django qual template deve ser renderizado e qual requisição está sendo tratada.
- **Organização em subpastas**: é recomendada para manter os arquivos organizados, especialmente em projetos com múltiplos apps.
- **`DIRS` em `settings.py`**: define os caminhos absolutos onde o Django procurará templates fora dos apps.

O que você estudou:

- O que são templates e seu papel na arquitetura do Django;
- Como criar e estruturar a pasta `templates`;
- Como configurar o Django para localizar seus templates;
- Como renderizar templates a partir de views.

Capítulo 12: Sintaxe de Templates

Introdução

No capítulo anterior, você aprendeu a renderizar um template HTML a partir de uma view no Django. Agora, vamos explorar de forma mais aprofundada o **sistema de templates do Django**, aprendendo a manipular dados dentro do HTML com variáveis, condicionais, loops e filtros.

Templates no Django não são apenas arquivos HTML estáticos, eles permitem a inserção de **lógica de exibição**, tornando as páginas dinâmicas e personalizáveis.

12.1 Utilizando Variáveis nos Templates

Uma das funcionalidades mais simples e poderosas dos templates é a possibilidade de **exibir variáveis** enviadas pelas views.

Exemplo: Saudação personalizada

Vamos revisar uma view que calcula uma mensagem com base no horário atual:

```
from django.shortcuts import render
from datetime import datetime

def saudacao(request):
    hora_atual = datetime.now().hour

    if 5 <= hora_atual < 12:
        mensagem = "Bom dia!"
    elif 12 <= hora_atual < 18:
        mensagem = "Boa tarde!"
    else:
        mensagem = "Boa noite!"

    return render(request, 'home.html', {'mensagem': mensagem})
```

Nesta view, estamos utilizando a função `render` para: - Renderizar o template `home.html`; - Passar a variável `mensagem` para esse template.

No arquivo `home.html`:

```
<h1>{{ mensagem }}</h1>
```

A sintaxe `{{ variável }}` permite exibir o valor de variáveis enviadas pela view. Neste caso, o Django substituirá `{{ mensagem }}` por "Bom dia!", "Boa tarde!" ou "Boa noite!", dependendo da hora da requisição.

12.2 Estruturas de Controle no Template

Condicional `if`

É possível controlar o conteúdo exibido usando instruções condicionais no HTML. Veja o exemplo:

```
<h1>{{ mensagem }}</h1>

{% if mensagem == "Bom dia!" %}
    <p>Que tal um café para começar o dia?</p>
{% elif mensagem == "Boa tarde!" %}
    <p>Uma água geladinha cairia bem agora.</p>
{% else %}
    <p>Durma bem!</p>
{% endif %}
```

- **`{% if %}`**: inicia a condição.
- **`{% elif %}`**: outra possibilidade (equivalente ao `elif` do Python).
- **`{% else %}`**: alternativa padrão.
- **`{% endif %}`**: finaliza o bloco condicional.

Repetição com `for`

A instrução `for` permite **iterar sobre listas** enviadas pela view.

Exemplo prático: lista de produtos Na view:

```
def produtos(request):
    lista = ["Notebook", "Mouse", "Teclado", "Fone", "Celular"]
    return render(request, 'produtos.html', {'produtos': lista})
```

No template `produtos.html`:

```
<h1>Lista de Produtos:</h1>
<ul>
    {% for produto in produtos %}
        <li>{{ produto }}</li>
    {% endfor %}
</ul>
```

Cada item da lista será renderizado como um item da `<ul>`.

12.3 Filtros no Template

Filtros servem para **modificar a exibição** de valores no template. Eles são aplicados com o caractere pipe (|).

Exemplos de filtros úteis

- **`upper`**: transforma texto em letras maiúsculas.

```
<h1>{{ mensagem|upper }}</h1>
```

- **`date`**: formata uma data.

```
<p>Data atual: {{ hoje|date:"d/m/Y" }}</p>
```

- **`length`**: retorna o tamanho de uma lista ou string.

```
<p>Total de produtos: {{ produtos|length }}</p>
```

Boas práticas com templates

- **Organização**: separe os arquivos de template em pastas com o nome do app.
- **Clareza**: evite lógica complexa nos templates. Deixe as regras mais elaboradas nas views.
- **Reutilização**: use herança de templates para criar páginas com layout consistente (isso será abordado nos próximos capítulos).

O que você estudou:

- Como passar variáveis das views para os templates com `{{ }}`;
- Como exibir conteúdo condicionalmente usando `{% if %}`;
- Como percorrer listas com `{% for %}`;
- Como usar filtros para modificar o conteúdo exibido nos templates.

Capítulo 13: Herança de Templates

Introdução

Ao construir aplicações web, é comum termos várias páginas com a **mesma estrutura**, como cabeçalhos, menus, rodapés e estilos visuais. Escrever esse conteúdo repetidamente em cada template não é eficiente. A solução para isso, no Django, é a **herança de templates**.

Neste capítulo, você aprenderá: - O que é herança de templates; - Como criar um template base; - Como utilizar blocos (`{% block %}`) para definir áreas personalizáveis; - Como criar templates filhos que herdam a estrutura principal.

13.1 Por que reutilizar templates?

À medida que um projeto web cresce, é comum que várias páginas compartilhem elementos em comum, como o cabeçalho, rodapé, menus de navegação e até estruturas visuais. Imagine um site com 10 páginas diferentes, todas com o mesmo menu superior. Se cada uma dessas páginas possuir um HTML independente com esse menu escrito manualmente, qualquer pequena alteração exigirá modificar o código em **todas** essas páginas — o que pode ser cansativo, demorado e propenso a erros.

É nesse cenário que a **reutilização de templates** se mostra essencial. Com ela, criamos um **template base** que serve como estrutura principal para o site. As demais páginas **herdam** esse layout e inserem apenas o conteúdo que varia. Isso torna o projeto muito mais **eficiente e organizado**.

13.2 O que é herança de templates?

A **herança de templates** permite que você crie um arquivo base (como um "molde") com o layout principal do site. Em seguida, outras páginas podem **herdar esse layout** e preencher apenas as partes que mudam, como o título ou o conteúdo principal.

Essa herança funciona por meio dos **blocos**, definidos com `{% block nome %}` e finalizados com `{% endblock %}`.

13.3 Configurando a pasta de templates no projeto

Antes de criar o template base, precisamos garantir que o Django saiba onde buscar os templates. No arquivo `settings.py`, ajuste a configuração `DIRS` dentro de `TEMPLATES`:

```
TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [os.path.join(BASE_DIR, 'templates')],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                # ...
            ],
        },
    },
]
```

Essa configuração aponta para a pasta `templates/` localizada na raiz do seu projeto. Dentro dela, podemos armazenar os arquivos base que serão usados por diferentes apps.

13.4 Criando o Template Base (`base.html`)

Passo 1: Estrutura da pasta

Crie a seguinte estrutura de diretórios:

```
templates/
└── base.html
```

Passo 2: Conteúdo do `base.html`

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
    <meta charset="UTF-8">
    <title>{% block title %}Meu Site{% endblock %}</title>
</head>
<body>
    <header>
        <h1>Bem-vindo ao Meu Site</h1>
    </header>

    <main>
        {% block content %}
        {% endblock %}
    </main>

    <footer>
        <p>&copy; 2025 Meu Site</p>
    </footer>
</body>
</html>
```

Explicação

- **`{% block title %}`** e **`{% block content %}`**: são blocos de conteúdo. Eles funcionam como pontos de substituição onde os templates filhos podem injetar conteúdo.
- O HTML define a estrutura básica comum a todas as páginas.

13.5 Criando um Template Filho

Exemplo: `index.html`

```
{% extends 'base.html' %}

{% block title %}Página Inicial{% endblock %}

{% block content %}
    <h1>{{ mensagem }}</h1>
    <p>Este é o meu primeiro template no Django.</p>
{% endblock %}
```

View correspondente (`views.py`)

```
from django.shortcuts import render

def index(request):
    mensagem = "Olá!"
    return render(request, 'index.html', {'mensagem': mensagem})
```

Rota no `urls.py`

```
from django.urls import path
from . import views

urlpatterns = [
    path('index/', views.index, name='index'),
]
```

13.6 Exemplo com Iteração: `produtos.html`

```
{% extends 'base.html' %}

{% block title %}Lista de Produtos{% endblock %}

{% block content %}
    <h1>Produtos:</h1>
    <ul>
```

```
        {% for produto in produtos %}
            <li>{{ produto }}</li>
        {% endfor %}
    </ul>
{% endblock %}
```

View correspondente

```
def produtos(request):
    lista = ["Notebook", "Teclado", "Mouse", "Fone", "Celular"]
    return render(request, 'produtos.html', {'produtos': lista})
```

13.7 Benefícios da Herança de Templates

- **Evita duplicação de código:** os elementos comuns são escritos uma única vez no template base e utilizados por várias páginas.
- **Facilita a manutenção:** ao modificar algo no layout geral, como a cor de fundo ou o rodapé, basta fazer isso no template base, e todas as páginas serão atualizadas automaticamente.
- **Acelera o desenvolvimento:** não é necessário reescrever a mesma estrutura HTML em todas as páginas.
- **Promove organização:** com blocos bem definidos, cada parte do projeto fica em seu devido lugar, facilitando o entendimento do código por outros desenvolvedores.

Reutilizar templates é uma prática fundamental no desenvolvimento web moderno, especialmente em aplicações que buscam escalabilidade e consistência visual.

O que você estudou:

- O conceito de herança de templates no Django;
- Como criar um `base.html` com estrutura comum;
- Como definir blocos personalizáveis;
- Como criar páginas que estendem o template base;
- Como manter um código HTML organizado e reutilizável.